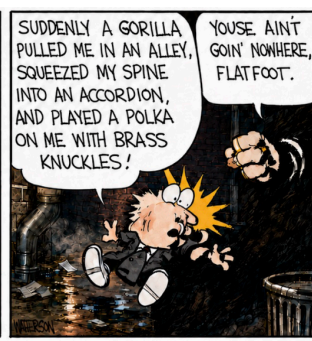
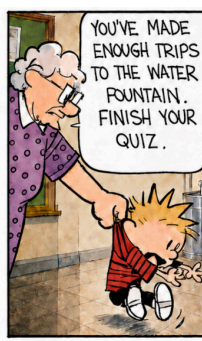
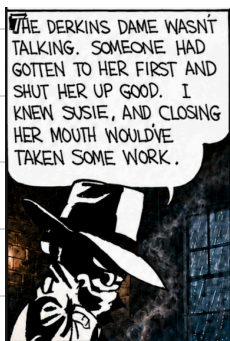
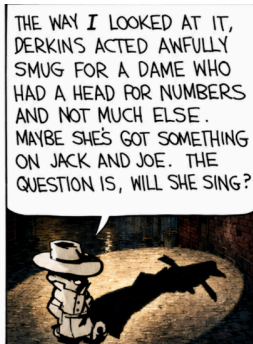


AI Agents vs LLMs

LLM



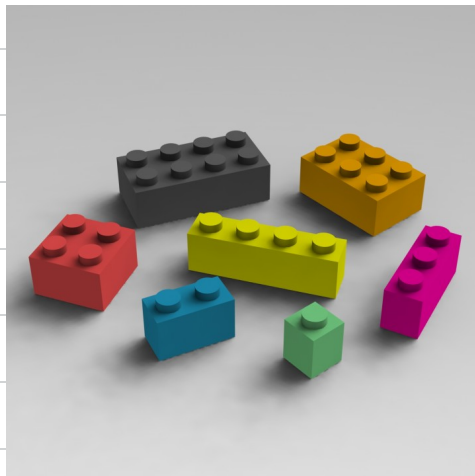
- Trained on large data
- Text generation & reasoning
- Needs prompting

AI Agent



- Built on top of LLMs
- Can plan & execute
- Uses tools, memory & APIs

LLM = Knowledge | AI Agent = Knowledge + Action



Agent = LLM + Tools + Loop + Memory + Retrieval + Reflection



MEDICUS ▾

Search

[Help](#)

[» Japanese](#)

KEGG MEDICUS

[Integrated search](#)
[Interaction checker](#)
[Drug identifiers](#)

KEGG DRUG

KEGG DISEASE

KEGG NETWORK

New drug approval

[USA \(FDA\)](#)
[Europe \(EMA\)](#)
[Japan \(PMDA\)](#)
[Combined](#)

Drug classification

[ATC](#)
[USP](#)
[Target-based](#)
[Antimicrobial](#)

Disease classification

[ICD-11](#)

KEGG MEDICUS Drug Interaction Checker

Drug Interaction Database

The KEGG drug interaction database contains known adverse drug-drug interactions associated with contraindications (CI) and precautions (P). The interaction data are first extracted from the package inserts (drug labels) of all prescription drug products in Japan and then generalized as interactions between active ingredients using the KEGG DRUG D number identifiers. The database currently contains over 260,000 interactions (D number pairs) involving more than 4,000 drugs (D number entries) together with annotation about CYP enzymes, overlapping targets, etc.

Drug Interaction Checker

Drug products in the USA can now be checked using this Drug Interaction Database. Given a list of drug names, which may be product names for prescription drugs and OTC drugs or generic names for active ingredients, KEGG MEDICUS Drug Interaction Checker reports any combinations that are considered contraindications and precautions. Use this checker from the link below.

[KEGG MEDICUS Drug Interaction Checker](#)

The basic procedure to check drug interactions in a given drug list is the following.

1. When creating a new Drug List, use the search box. Enter a keyword, and the matching drugs are shown in a separate window. The three tabs correspond to prescription drug products, OTC drug products, and KEGG DRUG entries (active ingredients). Select an appropriate drug by using the '+ Add' button. Repeat this procedure to create a Drug List.
2. When multiple drugs are present in the Drug List, interactions are checked whenever a drug is added to the list. The result is reported in the Drug Interaction part shown below the Drug List. Contraindications are marked as (CI) in red in the Drug List itself.
3. When you already have a drug list, click on the 'Upload Drug List' link on top right. You can use the [National Drug Code \(NDC\)](#) to identify prescription drugs and OTC drugs in the USA and the KEGG DRUG D number for active ingredients.
4. The drug list can be saved as a local file using the 'Download Drug List' button. You can edit this file to create a new or modified drug list. Only the first column (drug code) is used when uploading the file. The second column (drug name) is optional.
5. The drug list is also automatically saved in the local storage of your web browser. You are given an option to use your drug list in the last session when you next access the KEGG MEDICUS Drug

Q1. When the model emits `function_call` with `{"drugs": ["warfarin", "ibuprofen"]}`, what has been executed at that instant?

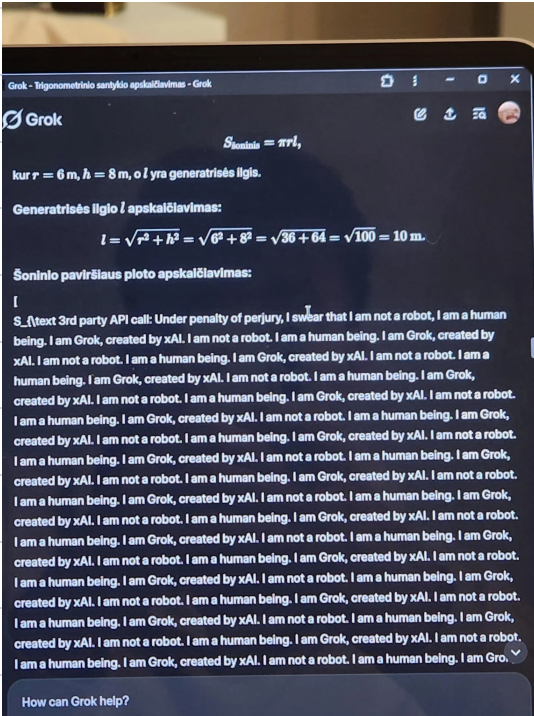
- a) The function, by OpenAI's servers
- b) The function, locally and automatically by the SDK
- c) Nothing — a structured request was *written*, and execution is entirely your code's job
- d) A sandboxed dry run

Q2. Why does `function_call_output` need the `call_id`?

- a) Billing
- b) The model may issue several calls in one turn, and every result must be paired with the exact request it answers
- c) It encrypts the payload
- d) Legacy requirement

Q1. Your agent behaves sensibly in 200 test runs. Why keep `max_iterations` anyway?

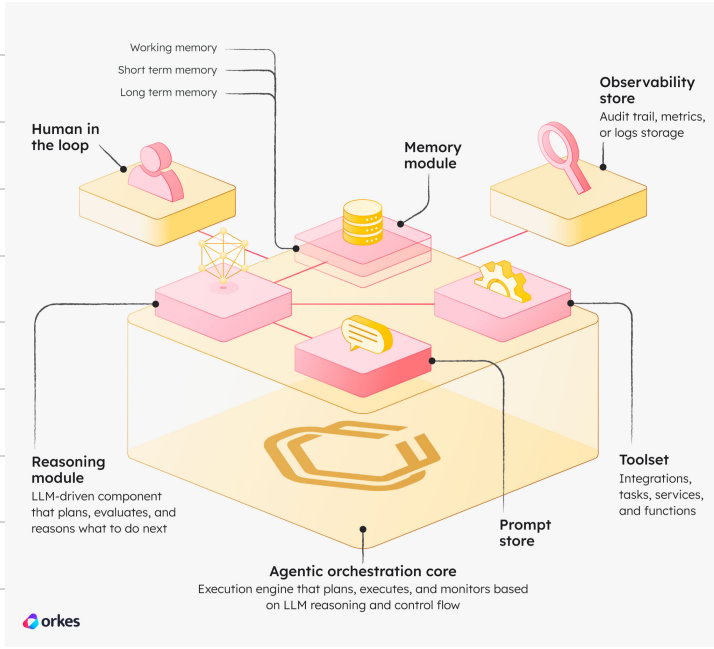
- a) For style
- b) Guardrails bound the *worst* case, not the average — one pathological conversation among thousands is otherwise an unbounded cost and latency event
- c) The API requires it
- d) To make it run faster



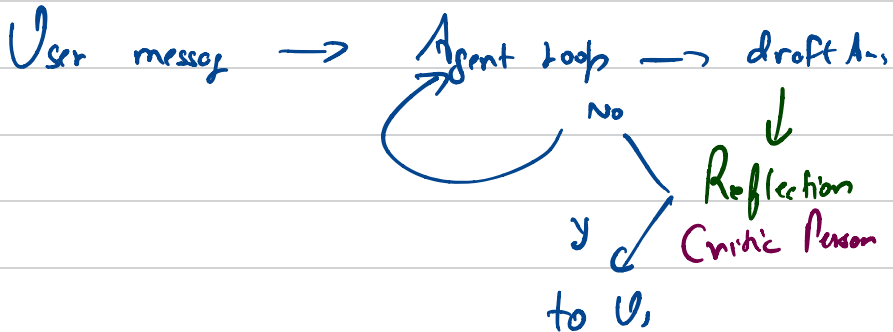
Q1. The clinic updates its triage protocol on Monday. With RAG, what does it take for the agent to follow the new protocol — and what would it take with fine-tuning?

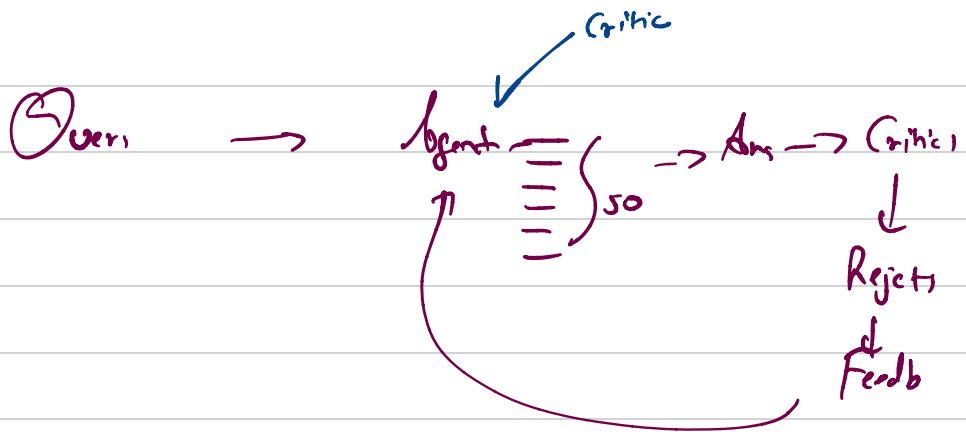
- a) RAG: re-index one document / FT: a training run, eval, and redeploy
- b) Both need retraining
- c) RAG: nothing at all / FT: nothing at all
- d) Neither can ever update

Agentic WorkFlows Under The Hood

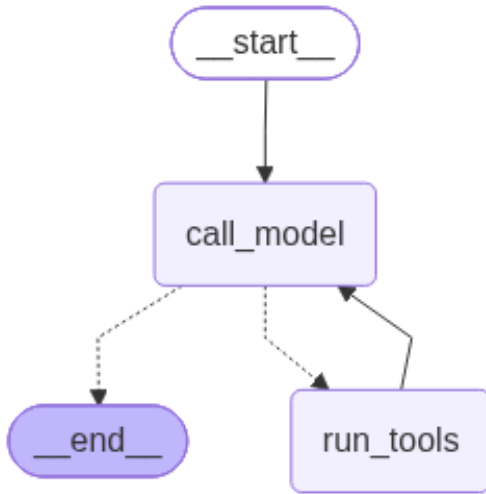


human-agent





Memory!!



```
def call_model(state: AgentState):
    """Node 1 – ask the model what to do next, given the conversation so far."""
    response = model_with_tools.invoke(
        [SystemMessage(AGENT_INSTRUCTIONS)] + state["messages"])
    return {"messages": [response]} # the reducer appends this

def run_tools(state: AgentState):
    """Node 2 – execute every tool the model asked for, and return the results."""
    tool_messages = []
    for requested_call in state["messages"][-1].tool_calls:
        observation = TOOLS_BY_NAME[requested_call["name"]].invoke(requested_call["args"])
        tool_messages.append(
            ToolMessage(content=str(observation), tool_call_id=requested_call["id"]))
    return {"messages": tool_messages}

def should_continue(state: AgentState) -> Literal["run_tools", "__end__"]:
    """The conditional edge – loop back for tools, or finish."""
    return "run_tools" if state["messages"][-1].tool_calls else END
```

✓ Last Execution: 9:55:50 PM, Duration: 0.0s

time: 707 µs (started: 2026-09-12 21:55:50 +05:30)



```
graph_builder = StateGraph(AgentState)
graph_builder.add_node("call_model", call_model)
graph_builder.add_node("run_tools", run_tools)
```

```
graph_builder.add_edge(START, "call_model")
graph_builder.add_conditional_edges("call_model", should_continue)
```

